

UNIVERSITAS INDONESIA
FAKULTAS ILMU KOMPUTER

Operating System (CSCM602055)

Session 7 : Sinkronisasi (Synchronization)

Hendrie Gunawan

Faculty of Computer Science, University of Indonesia

19 Sept 2026

Objective

- Understanding “Mengapa Sinkronisasi Diperlukan”
- Mendiagnosis Fenomena *Race Condition*
- Memvalidasi Solusi *Critical Section* (CS)
- Membedakan Primitif Sinkronisasi Hardware, Software dan High Level Tools

Mengapa Butuh Sinkronisasi?

- **Proses Kooperatif:** Proses atau *thread* yang berjalan bersamaan dapat saling memengaruhi atau dipengaruhi oleh proses lain.
- **Data Bersama (Shared Data):** Data yang dapat diakses oleh lebih dari satu proses secara bersamaan(konkuren)
- Jika beberapa proses **mengakses shared data**, bisa muncul **ketidak-konsistenan data (data inconsistency**).
- Untuk menjaga **konsistensi data**, dibutuhkan mekanisme agar proses bekerja **secara teratur (orderly execution)** dan tidak saling mengganggu ketika mengakses sumber daya bersama

Data Inkonsistensi

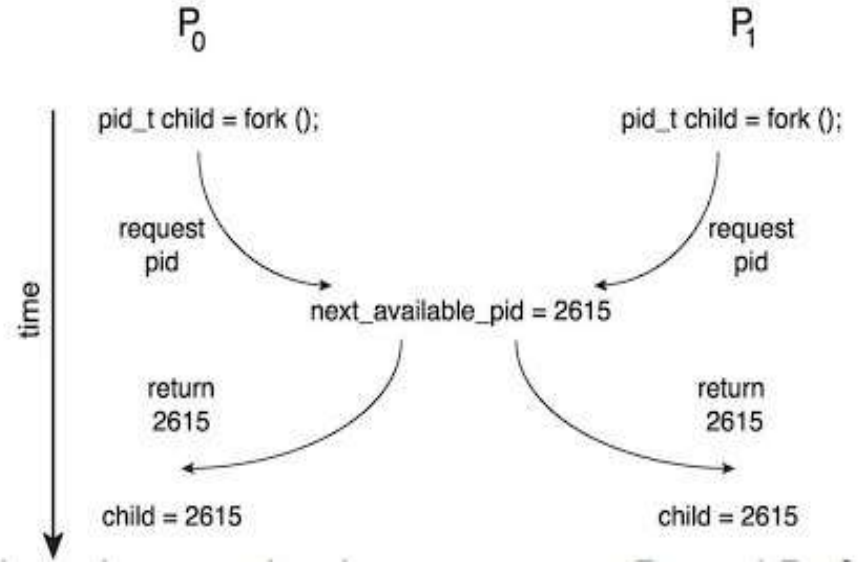
- Data inkonsistensi adalah kondisi di mana terdapat **perbedaan nilai** pada data yang sama di lokasi penyimpanan yang berbeda, atau ketika urutan perubahan data menghasilkan nilai yang secara logis salah
- Secara teknis, inkonsistensi data muncul karena operasi pada data tidak diselesaikan secara utuh sebelum proses lain mulai membaca atau mengubah data tersebut
- Contoh Skenario (Sistem Perbankan):
  - Bayangkan ada sebuah variabel global saldo = 1000.
  - Transaksi A: Menarik 200.
  - Transaksi B: Menarik 300.

Data Inkonsistensi

- Jika kedua transaksi berjalan bersamaan tanpa koordinasi:
  - A membaca saldo (1000).
  - B membaca saldo (1000).
  - A menghitung 1000 - 200 = 800.
  - B menghitung 1000 - 300 = 700.
  - A menyimpan kembali 800 ke memori.
  - B menyimpan kembali 700 ke memori.
  - Hasil Akhir: Saldo menjadi 700. Seharusnya, saldo adalah 500. Data sebesar 200 "hilang" karena terjadi inkonsistensi selama proses pembaruan.
- Terlihat kedua transaksi saling update data secara bersamaan sehingga secara abstrak seperti saling berlomba untuk mengakses dan mengupdate shared data. Kondisi ini disebut sebagai *Race Condition*

Race Condition

- *Race condition* terjadi ketika beberapa proses mengakses dan memanipulasi data yang sama secara bersamaan, dan hasil akhir dari eksekusi tersebut bergantung pada urutan akses yang tidak menentu
- Penyebab: Instruksi tingkat tinggi (seperti counter++) dipecah menjadi beberapa instruksi tingkat rendah di CPU (load, increment, store). Jika terjadi perpindahan konteks (context switch) di tengah-tengah urutan ini, data bisa saling update.
- Dampak: Inkonsistensi data serius. Misalnya, saldo bank yang tidak berkurang meskipun penarikan berhasil, atau data sensor yang hilang.



Critical Section

- Bagian kode dalam program dimana proses mengakses dan memodifikasi shared data.
- Struktur umum *Critical Section*:
  - **Entry Section**: proses meminta izin untuk masuk ke critical section.
  - **Critical Section**: proses mengakses/mengubah shared data.
  - **Exit Section**: proses keluar dari critical section.
  - **Remainder Section**: kode lain di luar bagian critical section.
- *Critical section problem* adalah masalah bagaimana mengatur agar hanya satu proses dalam satu waktu bisa berada di critical section.

```
while (true) {
        entry section
            critical section
        exit section
            remainder section
}
```

Critical Section Problem

- Sebuah solusi terhadap critical-section problem harus memenuhi tiga syarat berikut:
  - **Mutual Exclusion**: Jika proses P_i sedang dieksekusi di critical section-nya, maka tidak ada proses lain yang dapat dieksekusi di critical section mereka.
  - **Progress:** Jika tidak ada proses yang dieksekusi di *critical section* dan ada proses yang ingin masuk, maka pemilihan proses selanjutnya tidak boleh ditunda tanpa batas waktu
  - **Bounded Waiting:** Harus ada **batasan jumlah waktu** sebuah proses harus menunggu setelah meminta akses ke *critical section* sebelum permintaannya dikabulkan

Solusi Critical Section

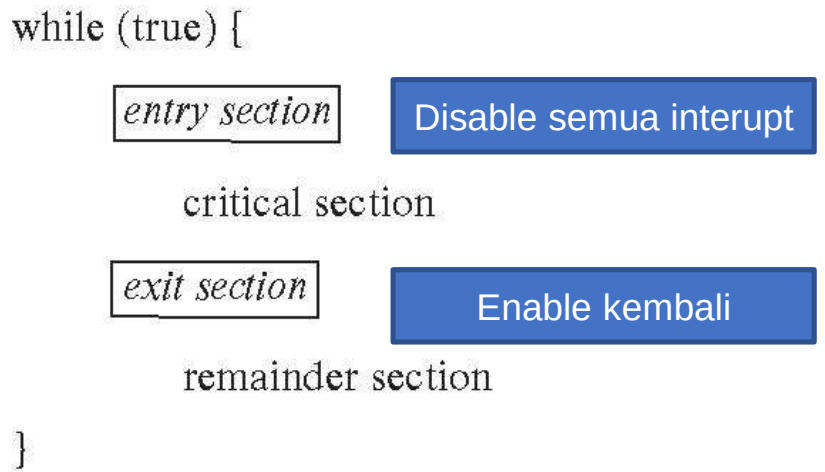
- Solusi Critical Section adalah mekanisme yang dirancang untuk mencegah **Race Condition** dan memastikan konsistensi data dengan mengatur urutan eksekusi proses/thread yang saling bersaing
- Secara hierarki, solusi ini dibagi menjadi tiga tingkatan:
  - **Solusi Hardware**,
  - **Software (Low-level)**, dan
  - **High-level Tools**

Solusi Hardware

- Solusi ini sangat cepat karena diimplementasikan langsung pada sirkuit CPU.
- Merupakan pondasi bagi semua solusi sinkronisasi lainnya, diantaranya
  - Interrupt based solution (Interrupt Disabling)
  - Atomic Instructions
  - Bus locking & Cache Coherency
  - Memory Barriers

Interupt Based Solution

- **Prinsip Utama** : mematikan interupsi sehingga kernel tidak bisa melakukan *preemption* (pengalihan tugas) saat proses berada di *critical section*
- **Kelebihan** :
  - Efektif di lingkungan single-core
  - Urutan instruksi dapat dijamin berjalan tanpa gangguan
- **Kelemahan** :
  - Jika **Critical Section Terlalu Panjang** akan berbahaya karena proses pengguna mematikan interupsi terlalu lama sehingga proses lain tidak bisa masuk, akibatnya sistem tidak responsive
  - **Starvation** dimana Beberapa proses mungkin tidak pernah mendapat giliran. Dapat menyebabkan hang dan melanggar prinsip fairness.
  - Tidak efektif pada sistem multi-core karena disabling interrupt hanya berlaku di satu CPU, tidak efektif untuk sistem multiprosesor



Atomic Instructions

- CPU harus menyediakan instruksi yang dijamin **atomik** (tidak bisa diinterupsi di tengah jalan), diantaranya :
  - **Test-and-Set (TSL):** Membaca nilai dari memori dan menulis nilai baru ke dalamnya dalam satu siklus memori yang tidak terputus
  - **Compare-and-Swap (CAS):** Di arsitektur x86, ini direpresentasikan dengan instruksi CMPXCHG dengan prefix LOCK. Ini adalah fondasi dari implementasi lock-free data structures.
    - CPU hanya akan menulis nilai_baru ke alamat jika isi saat ini di alamat sama dengan nilai_lama. Jika tidak, operasi gagal. Ini dilakukan dalam satu siklus memori yang tidak dapat diinterupsi.

```
boolean TestAndSet (boolean *target) {
    boolean rv = *target;
    *target = TRUE; // Set jadi terkunci
    return rv;      // Kembalikan nilai lama
}
```

- Jika rv adalah false, berarti lock berhasil didapat.
- Jika rv adalah true, berarti orang lain sedang memakai, kita harus mengulang.

```
int CompareAndSwap(int *value, int expected, int new_value) {
    int temp = *value;
    if (*value == expected)
        *value = new_value;
    return temp; // Mengembalikan nilai asli sebelum operasi
}
```

Bus Locking & Cache Coherency

- Pada arsitektur *multicore*, instruksi atomik saja belum cukup karena setiap core memiliki *cache* (L1/L2) sendiri.
  - The LOCK Prefix: Pada arsitektur x86, instruksi atomik biasanya disertai dengan prefix LOCK. Instruksi ini memerintahkan CPU untuk menegaskan sinyal pada Bus Memori.
  - Mekanisme: Saat sinyal LOCK aktif, akses ke alamat memori tersebut oleh prosesor lain akan diblokir secara fisik di level sirkuit hingga instruksi selesai.
  - Cache Locking: Pada CPU modern, alih-alih mengunci seluruh bus (yang sangat lambat), CPU menggunakan protokol cache coherency (seperti MESI). CPU "mengunci" satu baris cache (cache line) sehingga core lain tidak bisa memodifikasi salinan data tersebut secara bersamaan.

Memory Barriers

- CPU modern melakukan **Out-of-Order Execution** (menjalankan instruksi tidak berurutan demi performa)
- **Masalah:** CPU mungkin menjalankan instruksi penulisan data *setelah* instruksi pelepasan kunci, hanya karena instruksi tersebut dianggap lebih cepat selesai. Ini bisa menghancurkan logika sinkronisasi.
- **Solusi:** *Memory Barrier* adalah instruksi hardware yang memaksa CPU untuk menyelesaikan semua operasi memori yang tertunda sebelum melanjutkan ke instruksi berikutnya (memaksa prosesor untuk "mematuhi urutan“).
  - Load Barrier: Menjamin semua operasi baca selesai.
  - Store Barrier: Menjamin semua operasi tulis tersimpan di RAM.

Cara Kerja Memory Barriers

- Saat instruksi memory barrier dieksekusi, sistem memastikan semua operasi load dan store yang ada sebelum barrier selesai dieksekusi.
- Operasi load dan store yang ada setelah barrier tidak akan dimulai sebelum semua operasi sebelumnya selesai.
- Ini menjamin bahwa semua perubahan pada memori telah terlihat oleh prosesor lain

```
while (!flag)
   memory_barrier();
print x;
```

```
x = 100;
memory_barrier();
flag = true;
```

- Barrier di Thread 1 memastikan flag dibaca sebelum x dibaca.
- Barrier di Thread 2 memastikan x = 100 dieksekusi dan terlihat oleh memori sebelum flag = true.
- Catatan: Ini adalah operasi low-level yang cukup memperlambat dan biasanya hanya digunakan oleh kernel developer.

Solusi Software

- Menyelesaikan *critical section problem* untuk 2 proses tanpa hardware support menggunakan **shared variable turn** untuk mengatur giliran.
  - Nilai turn = 0 jika proses P_0 yang boleh masuk.
  - Nilai turn = 1 jika proses P_1 yang boleh masuk.
  - Awalnya turn diset ke ID proses yang boleh jalan duluan.
- Kelebihan:
  - Mutual Exclusion terpenuhi
  - Tidak perlu bantuan hardware
- Kekurangan:
  - Tidak memenuhi syarat Progress
  - Tidak menjamin Bounded Waiting
  - Hanya bekerja untuk dua proses

Shared Variable

```
int turn;          // 0 atau 1
```

Process Pi

```
while (true){
   while (turn = = j);

   /* critical section */ turn
   = j;
   /* remainder section */
}
```

Solusi Software

Shared Variable

```
int turn = 0;
```

Process Po (i=0, j=1)

```
while (true) {
 while (turn != 0);

   /* critical section */ turn
   = 1;
   /* remainder section */
}
```

Process P1 (i=1, j=0)

```
while (true) {
while (turn != 1);

   /* critical section */ turn
   = 0;
   /* remainder section */
}
```

- **Mengapa Atomic Load/Store Penting?**
  - **Tanpa atomic:** Operasi bisa diinterupsi di tengah, menyebabkan inconsistency.
  - **Dengan atomic:** Operasi dijamin selesai tanpa interupsi

Solusi Software : Peterson's Solution

- Menggunakan algoritma pemrograman untuk menjamin keamanan data
- Solusi ini secara spesifik dirancang untuk dua proses (P0 dan P1) yang mengeksekusi kode secara bergantian.
- Menggunakan dua variabel bersama:
  - int turn;
  - boolean flag[2];
- Asumsi: instruksi load dan store bersifat atomik (tidak dapat diinterupsi).
- turn → Menentukan giliran proses mana yang dapat masuk ke critical section.
  - Jika turn == i, maka proses Pi boleh masuk.
- flag[i] → Menandakan bahwa proses Pi siap masuk critical section.
  - Jika flag[i] = true, maka Pi sedang bersiap untuk masuk.

```
while (true){

        flag[i] = true;
        turn = j;
        while (flag[j] && turn = = j)
                ;

           /* critical section */

        flag[i] = false;

         /* remainder section */

}
```

Solusi Software : Peterson's Solution

- Solusi ini dianggap sempurna secara teoritis karena memenuhi tiga syarat utama *Critical Section*:
  - Mutual Exclusion : Hanya satu proses dapat berada di critical section karena turn tidak dapat bernilai i dan j sekaligus.
  - Progress : Jika tidak ada proses di critical section, proses yang siap tidak akan tertunda tanpa batas.
  - Bounded Waiting : Tiap proses akan mendapat giliran masuk critical section setelah maksimal satu kali giliran proses lain.
- Kelemahan:
  - **Tidak dijamin bekerja** pada prosesor modern karena prosesor dan compiler dapat melakukan instruction reordering untuk meningkatkan performa

Contoh Reordering :

Shared data

```
boolean flag = false;
int x = 0;
```

Thread 1 :

```
while (!flag)
  ;
print x
```

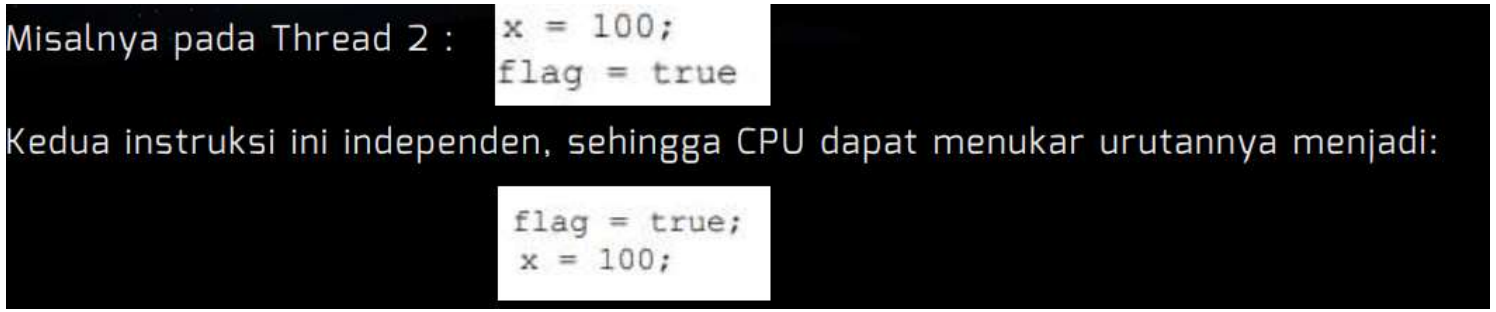
Thread 2 :

```
x = 100;
flag = true
```

- Ekspektasi: output 100
- Kemungkinan hasil nyata: 0, karena instruksi flag = true dan x = 100 bisa tertukar urutannya oleh CPU.

Peterson’s Solution on Modern Architecture

- Pada arsitektur modern, CPU dan compiler dapat mengubah urutan eksekusi instruksi (instruction reordering) untuk efisiensi.
- Hal ini dapat membuat dua proses masuk critical section secara bersamaan dan melanggar mutual exclusion.



- Jika ini terjadi, nilai output bisa menjadi salah (misalnya 0) dan Peterson’s Solution gagal menjamin eksklusi mutual. Untuk memastikan solusi bekerja diarsitektur modern, diperlukan Memory Barrier

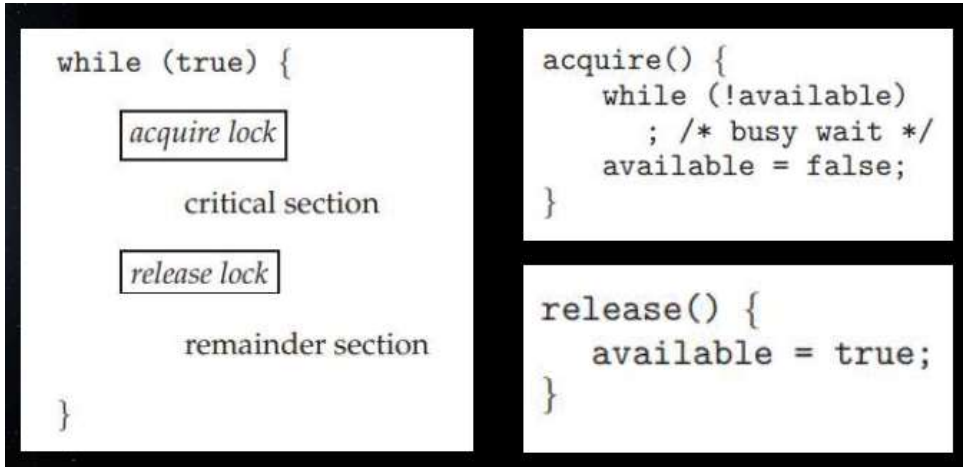
Solusi Software

- **Spinlocks (Busy-Waiting Primitives**) adalah Mekanisme di mana proses terus-menerus melakukan pengulangan (*looping/spin*) untuk memeriksa apakah kunci (*lock*) sudah tersedia
- **Karakteristik:** Menggunakan *busy-waiting*. Sangat efisien untuk *critical section* yang sangat pendek pada sistem multi-core karena tidak ada biaya *context switch*
- Pada OS modern, menidurkan thread (*blocking*) membutuhkan biaya besar (sekitar 1.000 - 5.000 siklus CPU) untuk menyimpan register, mengganti tabel halaman, dan memperbarui penjadwal. Spinlock **menghindari biaya ini**. Jika kunci dilepaskan dalam waktu 10 siklus, maka *spinning* jauh lebih murah daripada *blocking*.
- Kelemahan :
  - **CPU Burn:** Selama thread berputar, CPU bekerja 100% tanpa melakukan pekerjaan produktif.
  - **Tidak untuk Single-Core:** Pada sistem 1 CPU, Spinlock berbahaya. Jika thread A memegang kunci dan thread B berputar menunggu, thread A tidak akan pernah mendapatkan waktu CPU untuk melepaskan kunci tersebut (kecuali ada interupsi timer).

Solusi Software

- **Mutex (Mutual Exclusion) Lock:** Abstraksi penguncian yang paling umum. Jika kunci tidak tersedia, proses akan ditidurkan (*sleep/blocked*) oleh sistem operasi, membebaskan CPU untuk proses lain
- Mekanisme "Tidur" dan "Bangun" (Wait Queue)
  - Tidak seperti Spinlock yang berputar di CPU, Mutex terintegrasi dengan *Scheduler* OS.
    - **Gagal Lock**: Jika Thread A mencoba mengunci Mutex yang sudah dipegang Thread B, sistem operasi akan memanggil system call (misal: futex di Linux).
    - **Suspensi**: Kernel mengubah status Thread A menjadi BLOCKED, melepaskannya dari CPU, dan memasukkannya ke dalam Wait Queue (antrean tunggu) khusus milik Mutex tersebut.
    - **Sinyal**: Saat Thread B memanggil unlock(), kernel akan melihat Wait Queue, mengambil satu thread (misal Thread A), mengubah statusnya menjadi READY, dan memasukkannya kembali ke Ready Queue.

Mutex Lock



- Proses harus memanggil acquire() sebelum masuk Critical Section.
- Proses harus memanggil release() setelah keluar Critical Section.
- Panggilan acquire() dan release() harus atomik.

Mekanisme:

- Menggunakan satu variabel boolean available (tersedia/tidak).
- **acquire():** Jika **available**, acquire() berhasil dan available di-set ke false. Jika !available, proses akan diblokir (menunggu).
- **release():** Mengubah available kembali ke true .

Mutex Lock

- Kelemahan :
  - Busy waiting:
    - Kelemahan implementasi acquire() sebelumnya.
    - Proses yang menunggu looping (berputar) terus-menerus tanpa henti, mengecek ketersediaan.
    - Membuang-buang siklus CPU.
  - Spinlock:
    - Mutex lock yang menggunakan busy waiting.

- Kapan Spinlock berguna?
  - Keuntungan: tidak perlu context switch.
  - Sangat efisien di sistem multicore **HANYA** jika lock ditahan untuk durasi yang singkat.

Solusi Tingkat Tinggi (High-level Tools)

- Disediakan oleh pustaka pemrograman atau bahasa untuk mempermudah pengembang
- Berbeda dengan solusi *low-level* (seperti Spinlock) yang memaksa pengembang menangani instruksi CPU secara manual, solusi tingkat tinggi menawarkan **abstraksi** yang lebih aman, terstruktur, dan efisien.

Solusi Tingkat Tinggi (High-level Tools)

- Semaphore (Dijkstra's Primitives)
  - Semaphore adalah variabel integer yang digunakan untuk mengelola akses ke sumber daya bersama melalui sinyal. Ditemukan oleh Dijkstra tahun 1965
  - Mekanisme Internal (Tanpa *Busy-Waiting*)
    - Semaphore tidak menggunakan looping terus-menerus.
    - Jika sebuah proses melakukan wait() dan nilai Semaphore <= 0, proses tersebut akan diblokir dan dimasukkan ke dalam antrean tunggu (waiting queue).

Semaphore

Diakses melalui dua operasi utama (atomik) yaitu :

- wait(S) (atau P): Menunggu & mengurangi nilai.
- signal(S) (atau V): Menambah nilai.

wait(S) atau P(S):

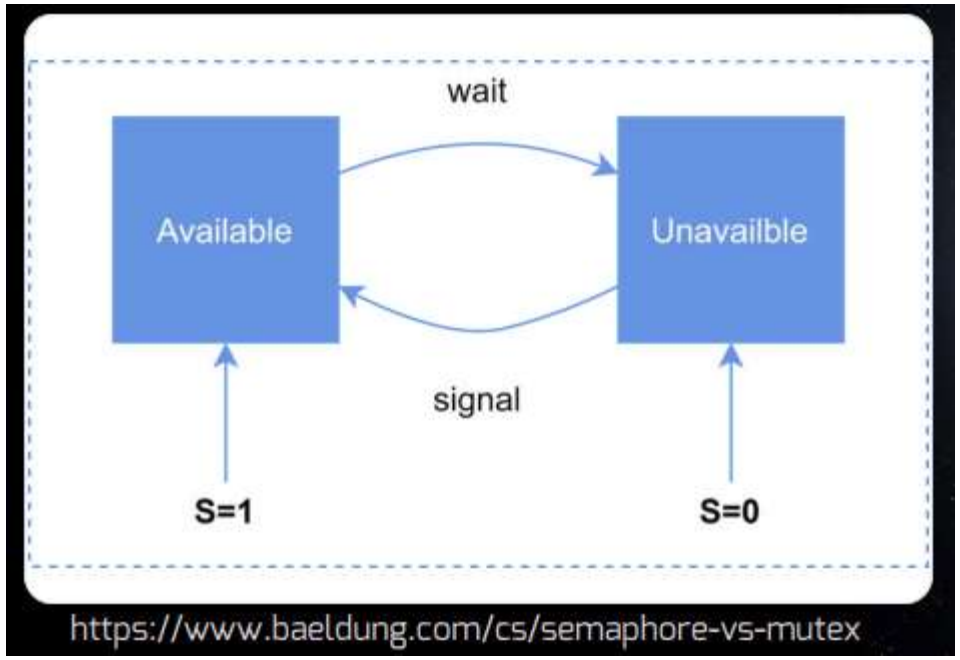
```
wait(S) {
    while (S <= 0)
      ; // busy wait
    S--;
}
```

signal(S) atau V(S):

```
signal(S) {
    S++;
}
```

Jenis-Jenis Semaphore

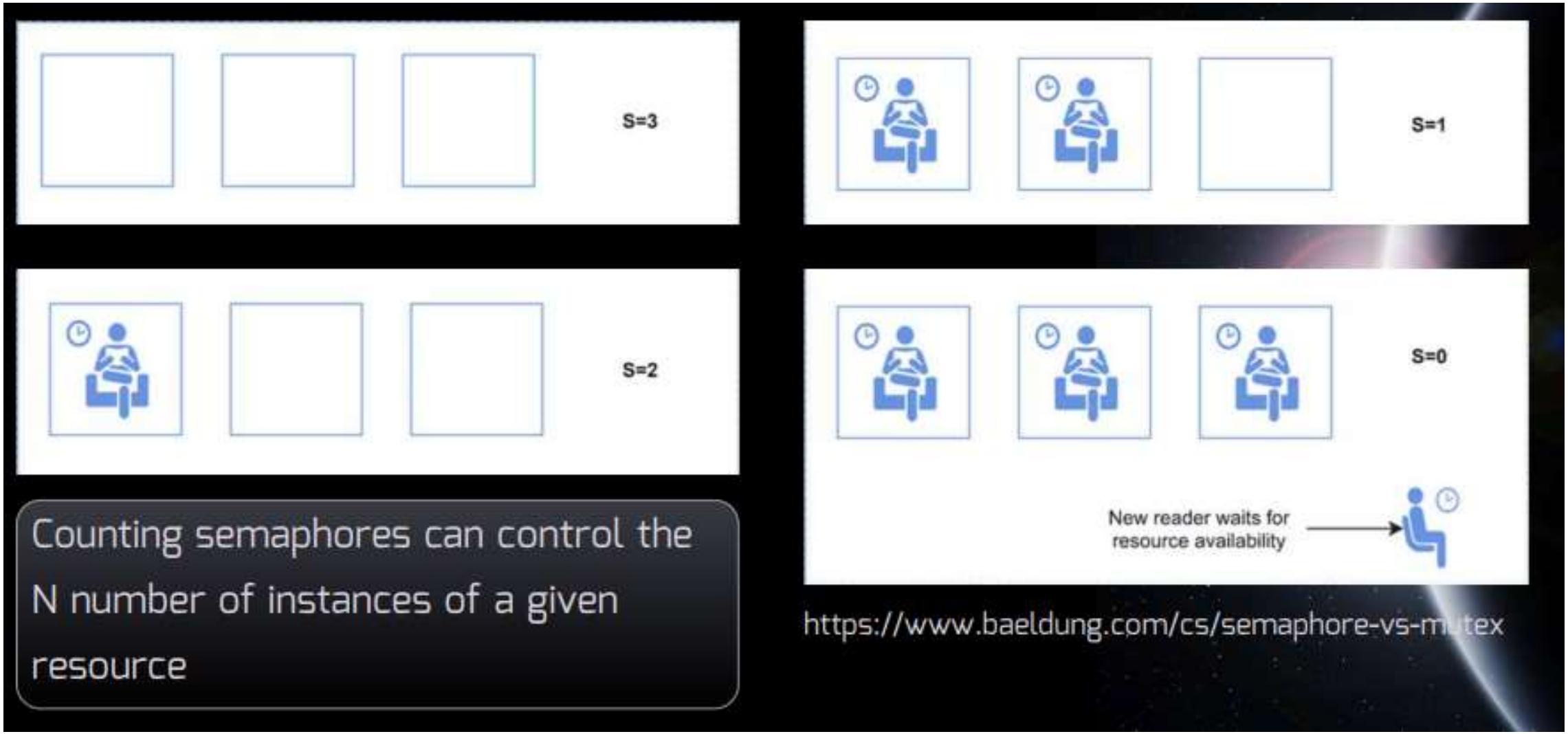
- **Binary Semaphore**: Nilainya hanya 0 atau 1.
  - Digunakan untuk Mutual Exclusion.
  - Perbedaannya dengan Mutex adalah Semaphore tidak memiliki konsep "kepemilikan" (ownership)—artinya, Thread A bisa membuka kunci yang dipasang oleh Thread B (meskipun ini praktik yang berbahaya).



Jenis-Jenis Semaphore

- **Counting Semaphore:** Nilainya bisa berapapun non-negatif.
  - Digunakan untuk mengelola sumber daya dengan jumlah terbatas (misalnya: membatasi maksimal 10 koneksi *database* secara simultan)
  - Semaphore diinisialisasi dengan jumlah resource yang tersedia.
  - Setiap proses yang ingin menggunakan resource akan melakukan operasi wait() pada semaphore (yang akan mengurangi hitungan).
  - Ketika suatu proses melepaskan resource, ia akan melakukan operasi signal() (yang akan menambah hitungan).
  - Ketika nilai semaphore menjadi 0, semua resource sedang digunakan.
  - Setelah itu, proses lain yang ingin menggunakan resource akan diblock sampai nilai semaphore menjadi lebih besar dari 0 lagi.

Jenis-Jenis Semaphore (Counting Semaphore)



Semaphore Usage

- Misalkan ada dua proses:
  - P1 menjalankan pernyataan S1
  - P2 menjalankan pernyataan S2
  - Kita ingin S2 dijalankan hanya setelah S1 selesai.
- Langkahnya:
  - Buat semaphore bersama bernama synch, diinisialisasi dengan nilai 0.
  - Di P1, tambahkan:

```
S1;
signal(synch);
```

  - → Artinya setelah S1 selesai, ia memberi sinyal bahwa “aku sudah selesai”.

- Di P2, tambahkan:

```
wait(synch);
S2;
```

- P2 akan menunggu sampai synch>0 (yakni setelah P1 melakukan signal()), baru kemudian mengeksekusi S2.

Problem with Semaphore

- Operasi wait() dan signal() pada semaphore juga menghadapi masalah busy waiting
- Ketika sebuah proses menjalankan operasi wait() dan menemukan bahwa nilai semaphore tidak positif (≤ 0), proses tersebut harus menunggu.
- Untuk mengatasi Busy Waiting,, proses tersebut akan menangguhkan dirinya sendiri (suspend)

Solusi Tingkat Tinggi (High-level Tools)

- Monitors (Hoare/Brinch Hansen), adalah Abstract Data Type (ADT) yang menyediakan:
  - Kumpulan variabel bersama (shared variables),
  - Fungsi/prosedur untuk mengakses variabel itu, dan
  - Mekanisme otomatis yang menjamin mutual exclusion.
- Monitor itu seperti “bungkus” disekitar kode dan data bersama, yang memastikan hanya satu proses bisa aktif di dalamnya pada suatu waktu

```
monitor monitor name
{
   /* shared variable declarations */

   function P1 ( . . . ) {
      . . .
   }

   function P2 ( . . . ) {
      . . .
   }

         .
         .
         .
   function Pn ( . . . ) {
      . . .
   }

   initialization_code ( . . . ) {
      . . .
   }
}
```

Kenapa Ada Monitors ?

Kasus 1: signal(mutex) dilakukan sebelum wait(mutex)

```
signal(mutex);
   ...
   critical section
   ...
wait(mutex);
```

Melanggar mutual exclusion karena critical section jadi bisa diakses beberapa proses sekaligus.

Kasus 2: Salah pemanggilan: wait(mutex) dipanggil dua kali

```
wait(mutex);
   ...
   critical section
   ...
wait(mutex);
```

Tidak ada pelanggaran eksklusivitas, tapi sistem macet (deadlock) karena proses nyangkut nunggu kunci yang dia sendiri pegang.

Kasus 3: lupa panggil wait(mutex) atau signal(mutex)

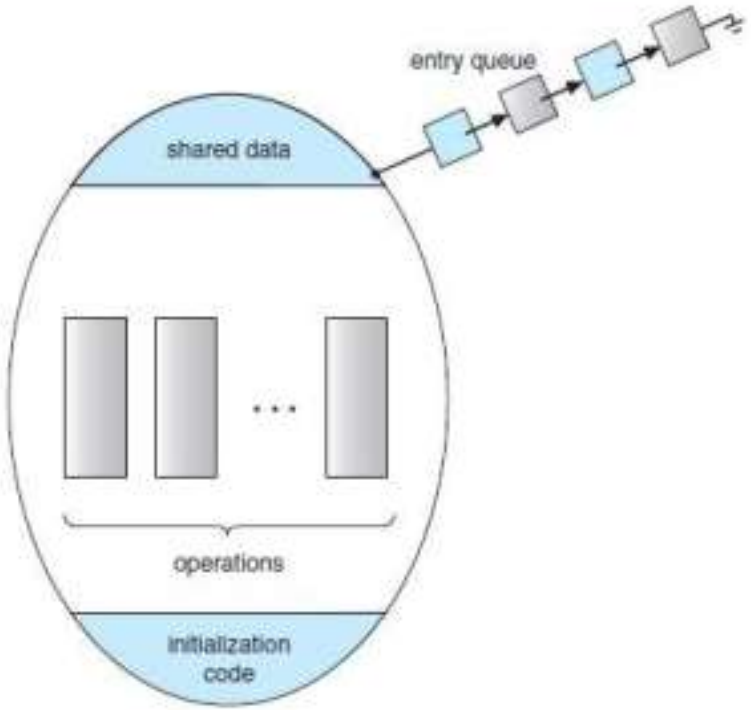
```
// lupa wait(mutex)
critical_section();
signal(mutex);
```

```
wait(mutex);
critical_section();
// lupa signal(mutex)
```

Mengabaikan salah satu operasi bisa bikin sistem tidak konsisten (race condition) atau berhenti total (deadlock).

Monitors : Condition Variable

- Antrean untuk proses-proses yang sedang menunggu (suspended)sebuah kondisi tertentu di dalam monitor
- **Condition Variables** memiliki 2 operasi utama yaitu:
  - x.wait():
    - Proses yang memanggil ini akan segera ditidurkan dan melepaskan hak akses eksklusifnya atas Monitor agar proses lain bisa masuk.
    - Proses ini dimasukkan ke antrean tunggu milik kondisi x
  - x.signal():
    - Membangunkan (resume) tepat satu proses yang sedang menunggu pada variabel kondisi x.
    - Proses yang dibangunkan akan mencoba mengambil lock monitor kembali agar bisa melanjutkan eksekusinya



Monitors signal vs Semaphore Signal

- **Monitor Signal (x.signal())**
  - Stateless (Tidak Mengingat).
  - Hanya punya efek jika ada proses yang sedang wait() di kondisi x.
  - Jika x.signal() dipanggil tapi tidak ada proses yang menunggu di x, maka signal() itu akan hilang begitu saja (tidak terjadi apa-apa).

- **Semaphore Signal (V())**
  - Stateful (Mengingat).
  - Selalu mengubah nilai semaphore (misalnya, S++).
  - Jika signal() dipanggil sebelum wait(), nilai S akan bertambah, dan wait() berikutnya mungkin tidak akan bloking

Implementasi Monitor & Masalah Urutan Resuming

- **Implementasi Monitor**
  - Monitor dapat diimplementasikan menggunakan semaphore. Komponen utamanya:
    - Satu semaphore mutex (nilai 1) untuk mutual exclusion monitor.
    - Satu semaphore next (nilai 0) dan next_count untuk proses yang melakukan signaling.
    - Satu semaphore (misal x_sem) dan counter (misal x_count) untuk setiap condition variable x

- **Masalah Urutan Resuming**
  - Jika x.signal() dipanggil dan banyak proses menunggu di x, siapa yang harus bangun?
    - Solusi standar: FCFS (First-Come, First-Served).
    - Solusi Lanjutan: Conditional x.wait(c). c adalah nomor prioritas.
    - Saat x.signal() dipanggil, proses dengan prioritas c terkecil yang akan dibangunkan.

Masalah Tools Sinkronisasi

- Penggunaan tools sinkronisasi (mutex, semaphore, monitor) dapat menimbulkan masalah baru yaitu Liveness Failure.
- Liveness adalah properti sistem yang memastikan setiap proses dapat membuat kemajuan (make progress) dalam siklus eksekusinya.
- Kegagalan Liveness (Liveness Failure) Adalah Situasi di mana sebuah proses menunggu tanpa batas waktu (indefinite waiting) dan gagal membuat kemajuan. Proses ini tidak mati, tapi macet. Contoh umum:
  - Infinite Loop (paling sederhana).
  - Deadlock.
  - Starvation.
  - Priority Inversion yaitu Sebuah proses berprioritas TINGGI terpaksa menunggu sebuah proses berprioritas RENDAH
    - Solusi : Priority Inheritence Protocol yaitu proses yang berprioritas lebih rendah mewarisi prioritas dari proses berprioritas lebih tinggi yang menunggunya

Contoh Priority Inversion Problem

- Terdapat 3 Proses dengan Prioritas: H (Tinggi) > M (Medium) > L (Rendah), Resource: S (misal: mutex lock).
  - Proses L (Rendah) mendapatkan lock S.
  - Proses H (Tinggi) ingin memakai S. H bloking (menunggu L).
  - Proses M (Medium) siap untuk dieksekusi.
  - Scheduler CPU melihat proses yang siap: M dan L.
  - Karena Prioritas(M) > Prioritas(L), scheduler menjalankan M (melakukan preempt pada L).
  - Maka H (Tinggi) sekarang menunggu M (Medium) selesai, agar L (Rendah) bisa jalan dan melepaskan lock. Ini yang disebut dengan **Priority Inversion**.

Contoh Priority Inheritence Protocol

- Terdapat 3 Proses dengan Prioritas: H (Tinggi) > M (Medium) > L (Rendah), Resource: S (misal: mutex lock).
  - L (Rendah) memegang lock S.
  - H (Tinggi) bloking, menunggu S.
  - Sistem operasi melihat ini dan mewariskan prioritas H ke L.
    - Prioritas L sekarang = Prioritas H (untuk sementara).
  - Proses M (Medium) siap dieksekusi.
  - Scheduler CPU melihat proses yang siap: M dan L (yang sudah diwarisi).
  - Karena Prioritas(L [warisan]) > Prioritas(M), L tidak bisa di-preempt oleh M.
  - L berjalan, selesai memakai S, melepaskan lock, dan prioritasnya kembali normal.
  - H (Tinggi) akhirnya mendapatkan lock S dan bisa berjalan.


Apa ada pertanyaan?


JANGAN SUSAH" PERTANYAANMU